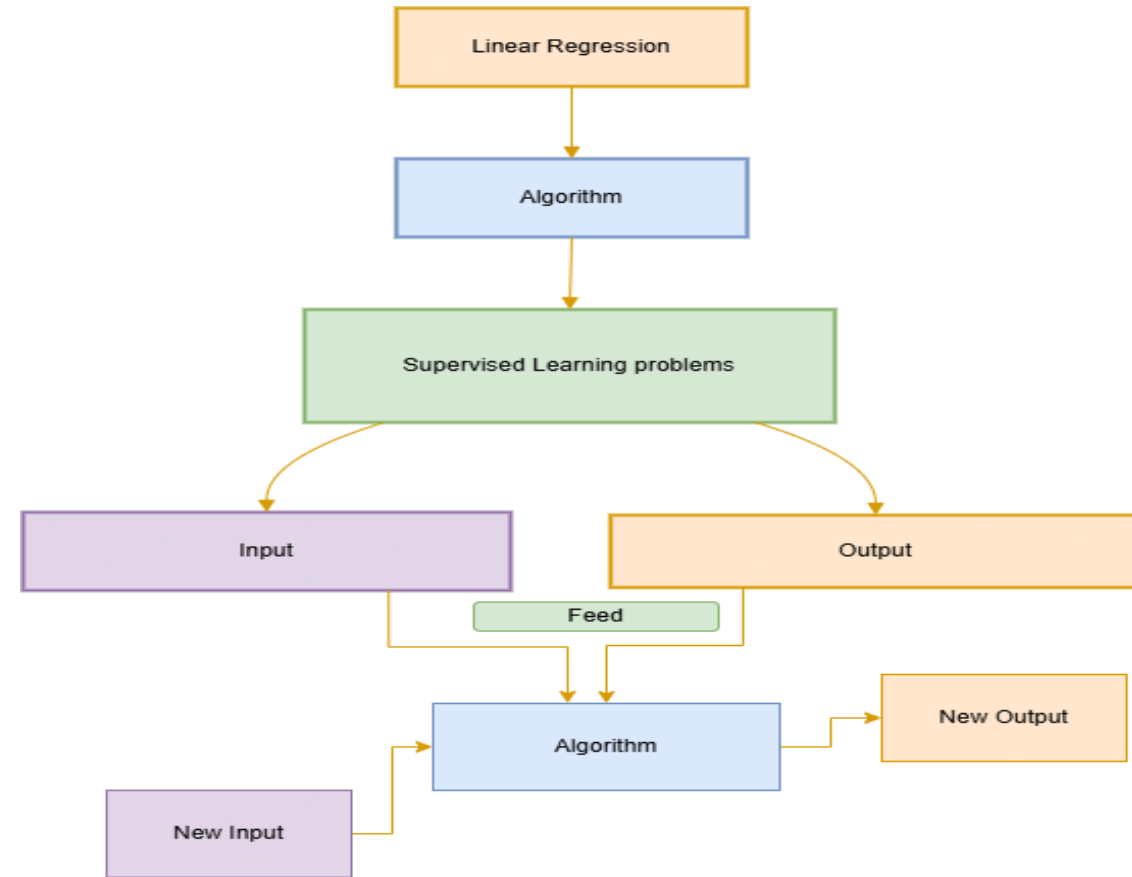


Linear Regression and Its Extensions

Dr. Prasenjit Dey,
Assistant Professor,
NIT Rourkela,
Odisha, India

Linear Regression

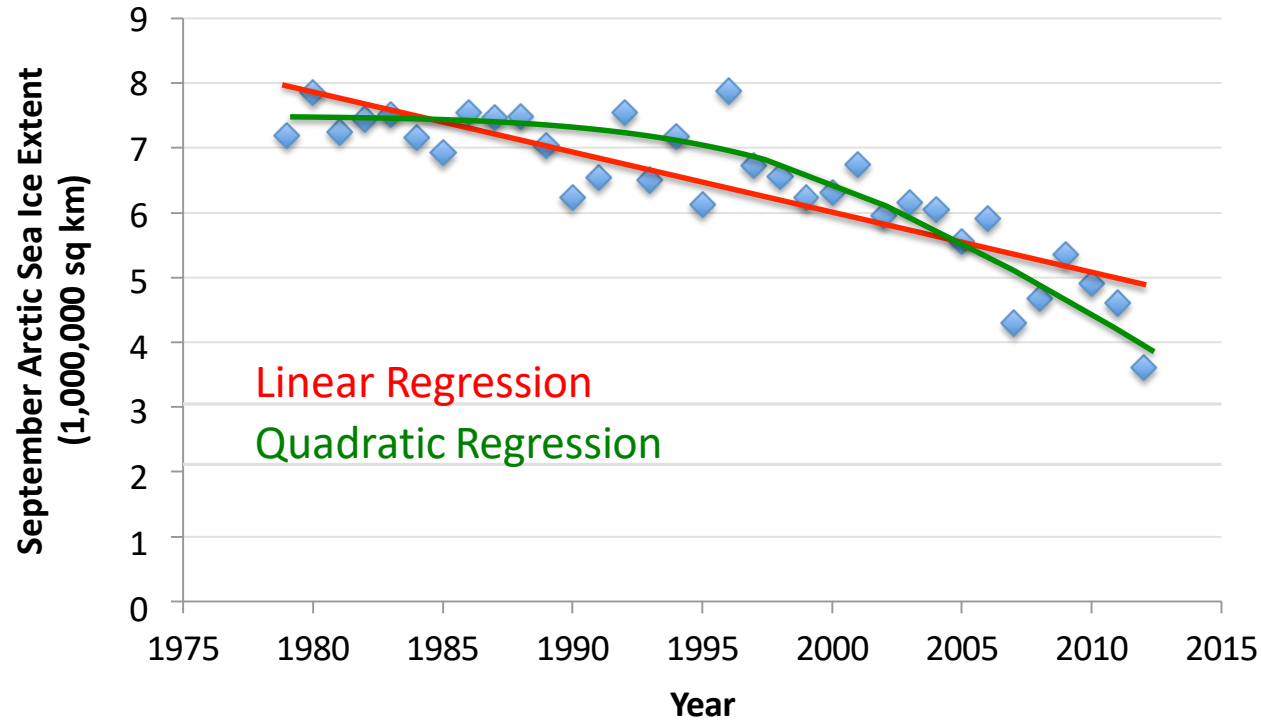
- A machine learning algorithm, used to solve supervised learning problems where:
 - We have input (X) and corresponding output (y).
 - The goal is to predict y from X .
- Linear regression is like fitting a line or (hyper)plane to a set of points.
- **How Does It Work?**
 - Feed input and output data to an algorithm
 - The algorithm learns the relationship (pattern) between input and output
 - Once trained, the algorithm can predict output for new input



Regression

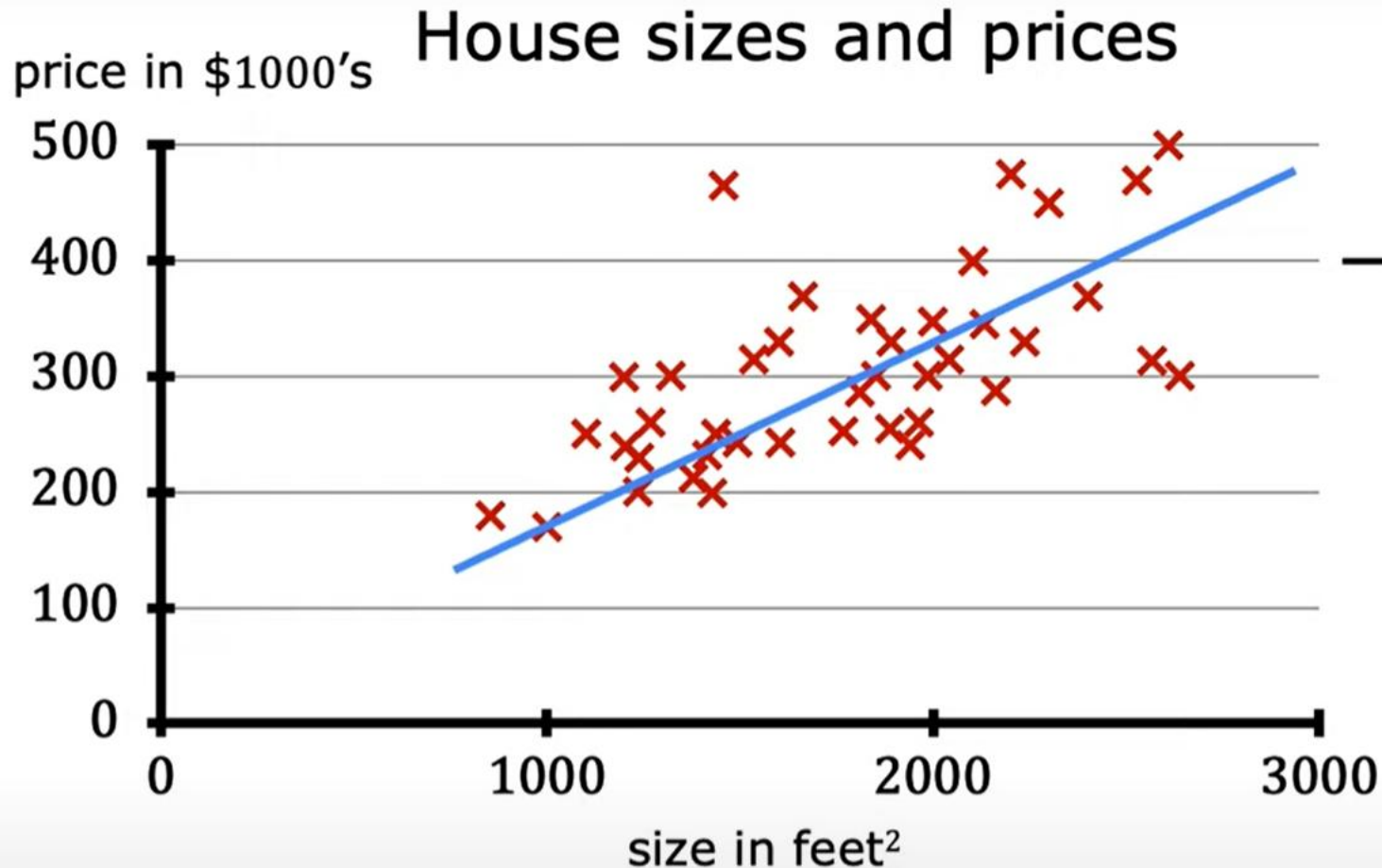
Given:

- Data $X = \{x^{(1)}, \dots, x^{(n)}\}$ where $x^{(i)} \in \mathbb{R}^d$
- Corresponding labels $y = \{y^{(1)}, \dots, y^{(n)}\}$ where $y^{(i)} \in \mathbb{R}$



Data from G. Witt. Journal of Statistics Education, Volume 21, Number 1 (2013)

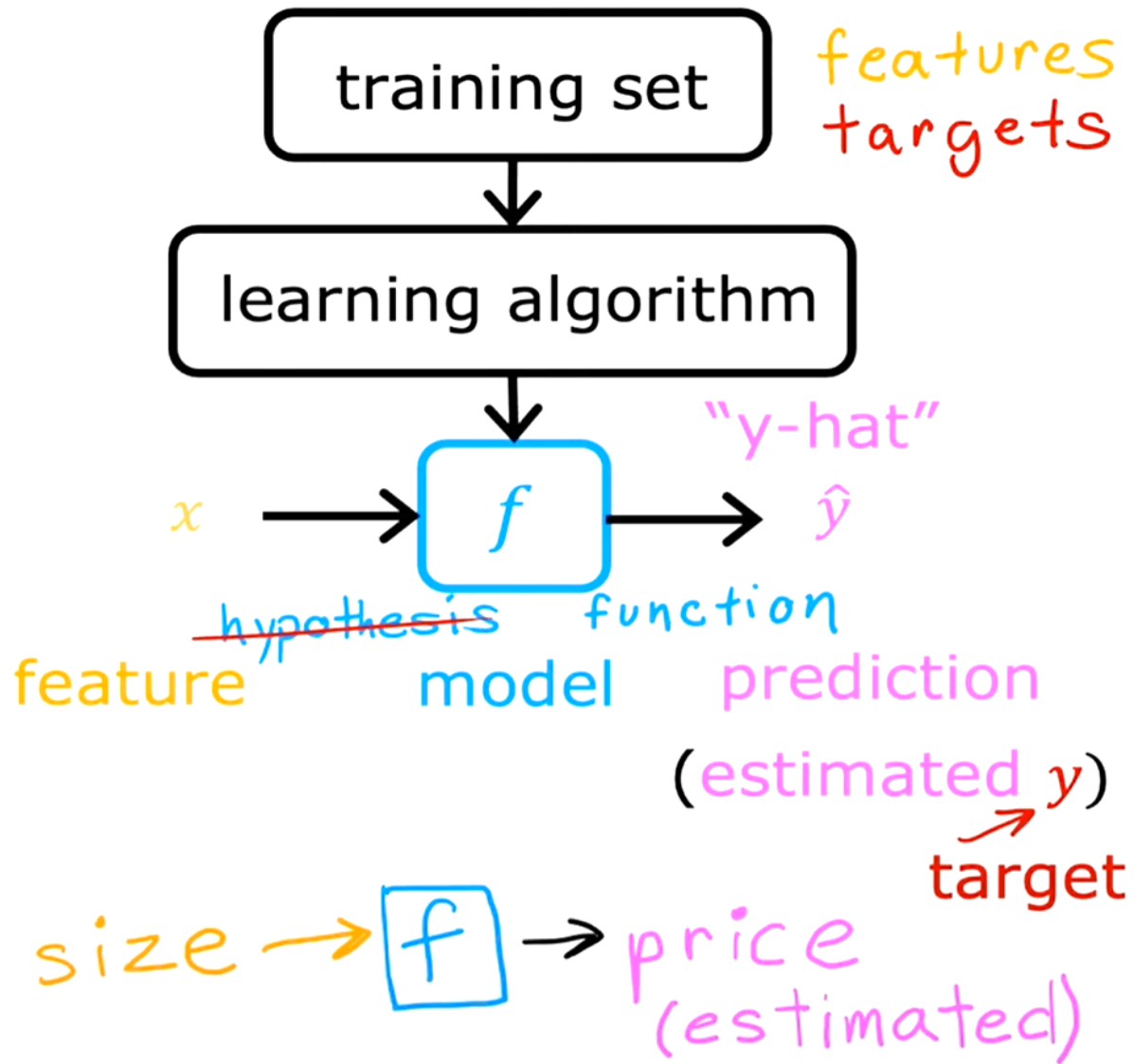
Linear Regression: Straight-line fitting



Data table

size in feet ²	price in \$1000's
2104	400
1416	232
1534	315
852	178
...	...
3210	870

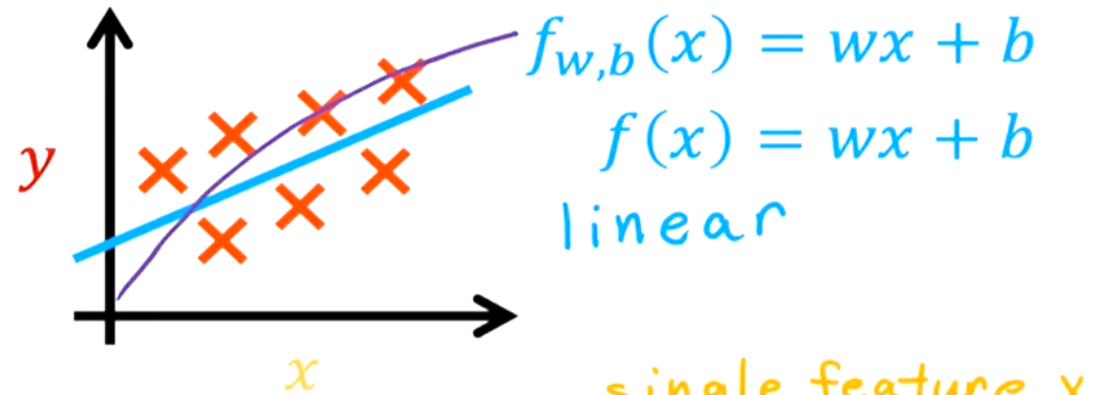
Linear regression with one variable



How to represent f ?

$$f_{w,b}(x) = wx + b$$

$f(x)$



Linear regression with **one** variable.
size

Univariate linear regression.
one variable

Types of Linear Regression

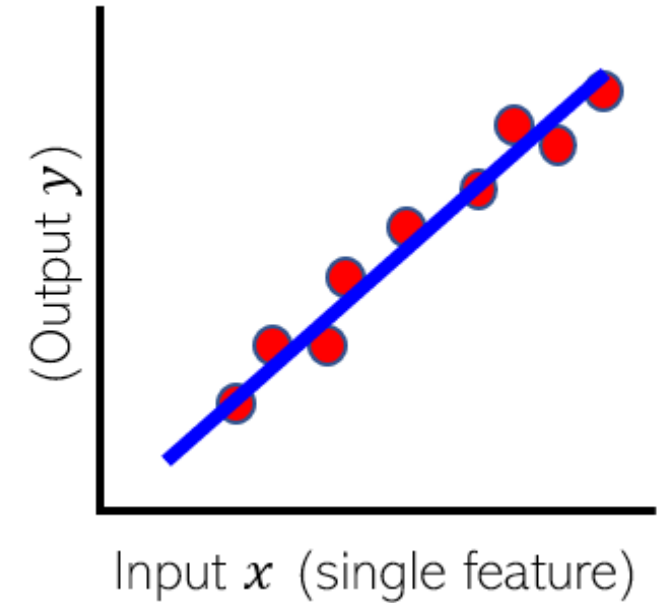
Univariate Linear Regression

- Used when we want to predict a target value (dependent variable) using only one input feature (independent variable).
- It assumes a straight-line relationship between the two.

$$y = b + wx + \epsilon$$

Where,

- y is the actual observed output (dependent variable)
- x is the input (independent variable)
- b is the intercept (value of $\hat{y}$ when $x=0$)
- w is the slope or coefficient (how much y changes with 1 unit change in x)
- ϵ is the error term (accounts for noise, randomness)



Types of Linear Regression

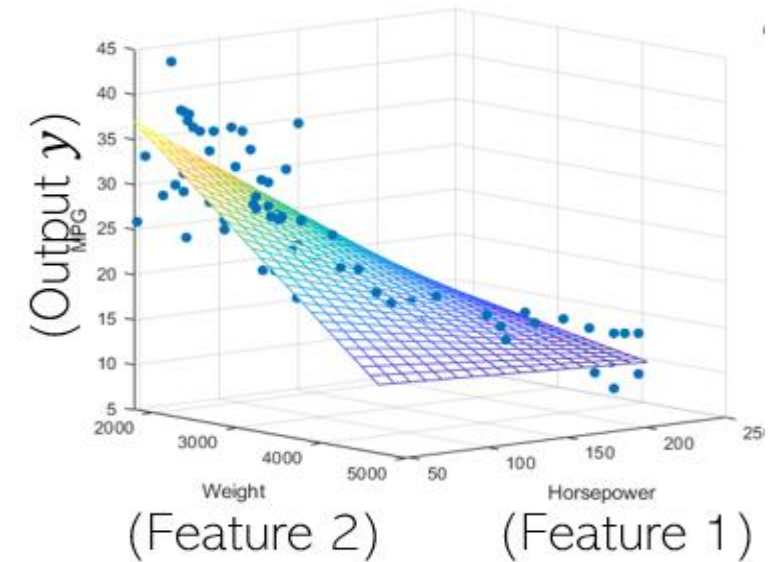
Multivariate Linear Regression

- Involves more than one independent variable to predict a dependent variable.
- The equation is:

$$y = b + w_1 x_1 + w_2 x_2 + \dots + w_n x_n + \epsilon$$

Where,

- y is the actual observed output (target variable)
- $X = [x_1^T, x_2^T, \dots, x_n^T]$ are the independent (input) variables
- b is the intercept (bias)
- $w_1, w_2, \dots, w_n$ are the coefficients (weights for each input feature)
- ϵ is the error term (accounts for noise, randomness)



Linear Regression: Pictorially

Q. What if a straight line or plane doesn't model the input-output relationship well (e.g., when the data has a nonlinear pattern)?

A. Even if the data shows a nonlinear relationship, linear models are not useless. We can still use them by transforming the input features.

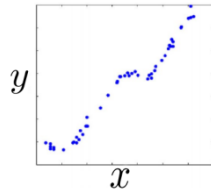
Q. How can a linear model fit a curve or nonlinear surface?

A. We can even fit a curve using a linear model after suitably transforming the inputs

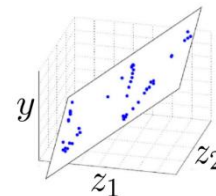
- The transformation $\phi(\cdot)$ can be predefined or learned (e.g., using kernel methods or a deep neural network based feature extractor).

$$y \approx \mathbf{w}^T \phi(\mathbf{x})$$

$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \phi(x)$$



Original (single) feature
Nonlinear curve needed



Two features
Can fit a plane (linear)

Evaluation Metrics

1. Mean Absolute Error (MAE)

Measures the average absolute difference between the actual values and the predicted values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Where:

- n is the number of data points
- y_i is the actual value
- $\hat{y}_i$ is the predicted value

2. Mean Squared Error (MSE)

Measures the average squared difference between the actual values and the predicted values. It penalizes larger errors more than MAE.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Evaluation Metrics for Linear Regression

3. Root Mean Squared Error (RMSE)

The square root of MSE, gives error in the same unit as the target variable.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

4. R-squared (R^2 or Coefficient of Determination)

Represents the proportion of the variance in the dependent variable that is predictable from the independent variables. It ranges from 0 to 1 (higher is better).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Where,

- $\bar{y}$ is the mean of the actual values

Evaluation Metrics for Linear Regression

5. Adjusted R-squared (Adjusted R^2)

Adjusted R^2 modifies the R^2 value by accounting for the number of predictors in the model. It **penalizes** the addition of unnecessary predictors and is more reliable for comparing models with different numbers of independent variables.

Formula:

$$R_{adj}^2 = 1 - (1 - R^2) \cdot \frac{n - 1}{n - k - 1}$$

Where:

- R^2 is the regular R-squared
- n is the number of observations
- k is the number of independent variables (predictors)

Loss Functions for Regression

- Many possible loss functions for regression problems

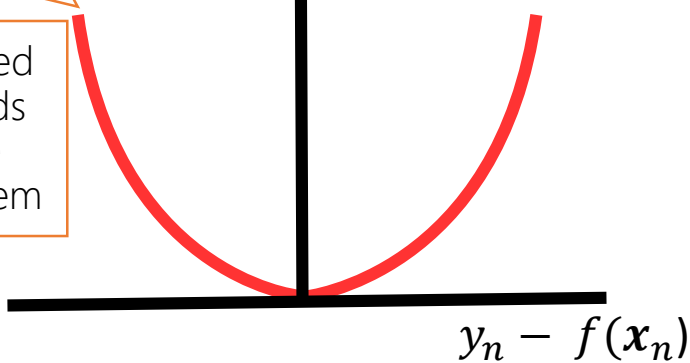
Choice of loss function usually depends on the nature of the data. Also, some loss functions result in easier optimization problem than others



Squared loss

Loss $(y_n - f(x_n))^2$

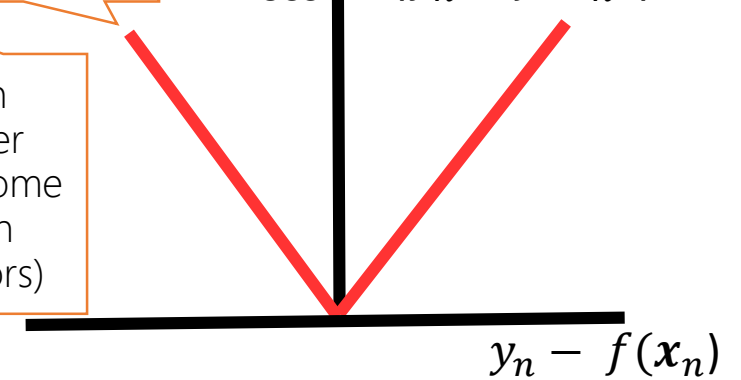
Very commonly used for regression. Leads to an easy-to-solve optimization problem



Absolute loss

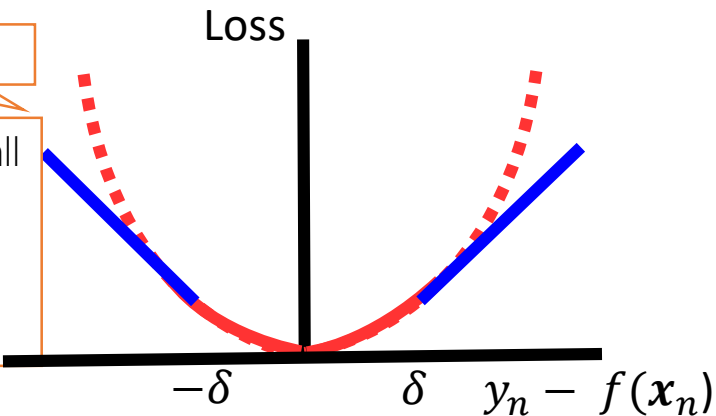
Loss $|y_n - f(x_n)|$

Grows more slowly than squared loss. Thus better suited when data has some outliers (inputs on which model makes large errors)



Huber loss

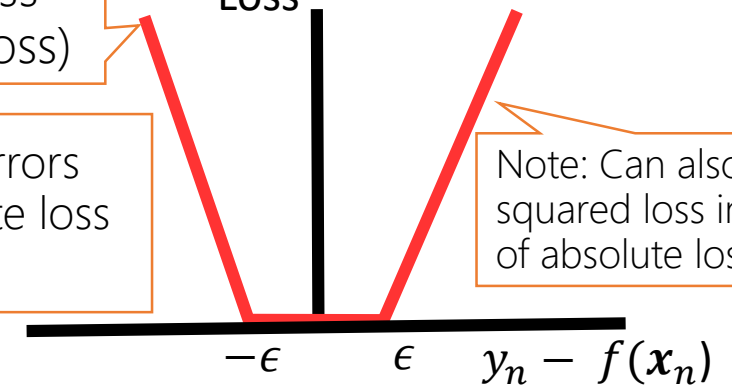
Squared loss for small errors (say up to δ); absolute loss for larger errors. Good for data with outliers



ϵ -insensitive loss (a.k.a. Vapnik loss)

Zero loss for small errors (say up to ϵ); absolute loss for larger errors

Loss $|y_n - f(x_n)| - \epsilon$

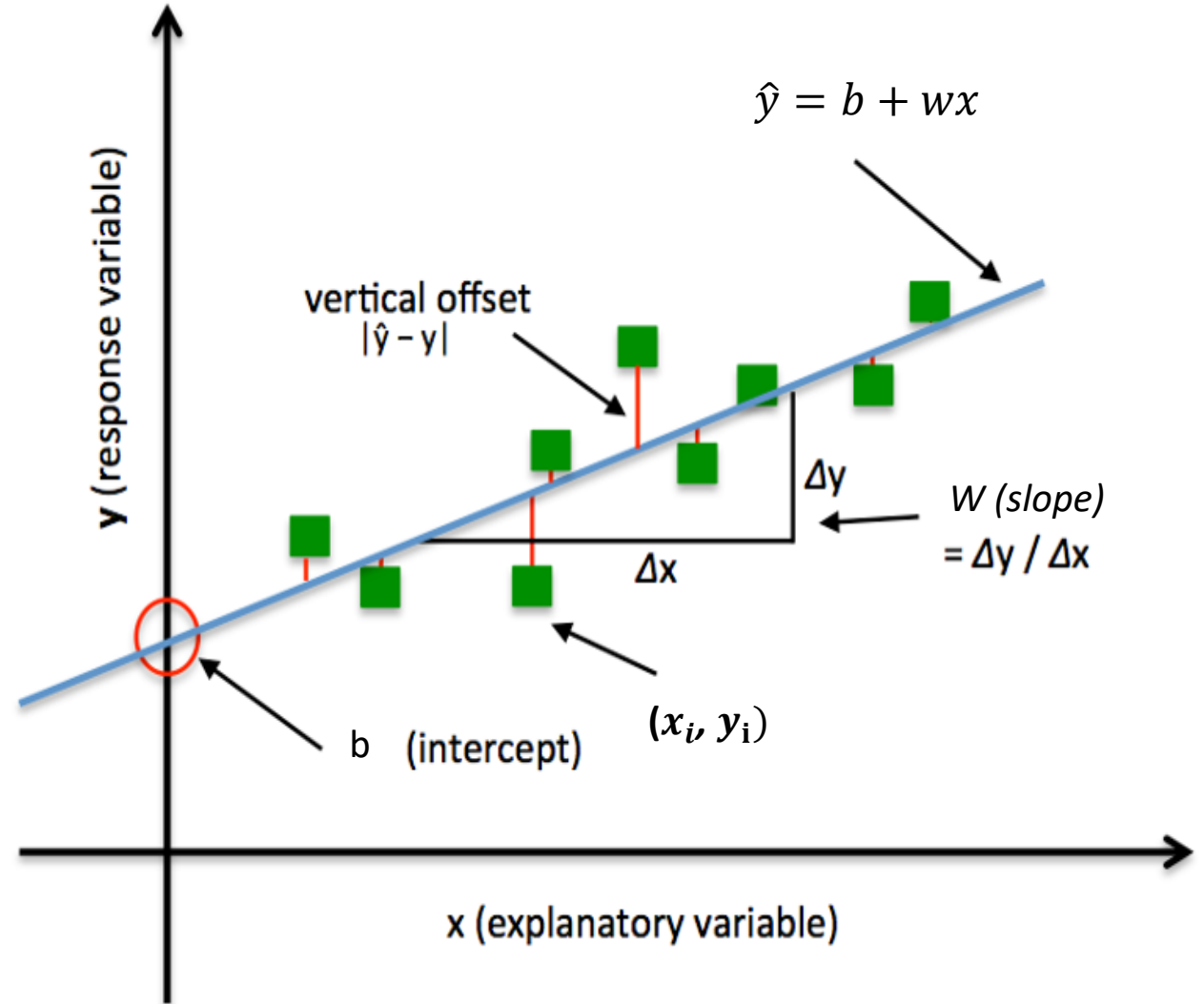


Note: Can also use squared loss instead of absolute loss

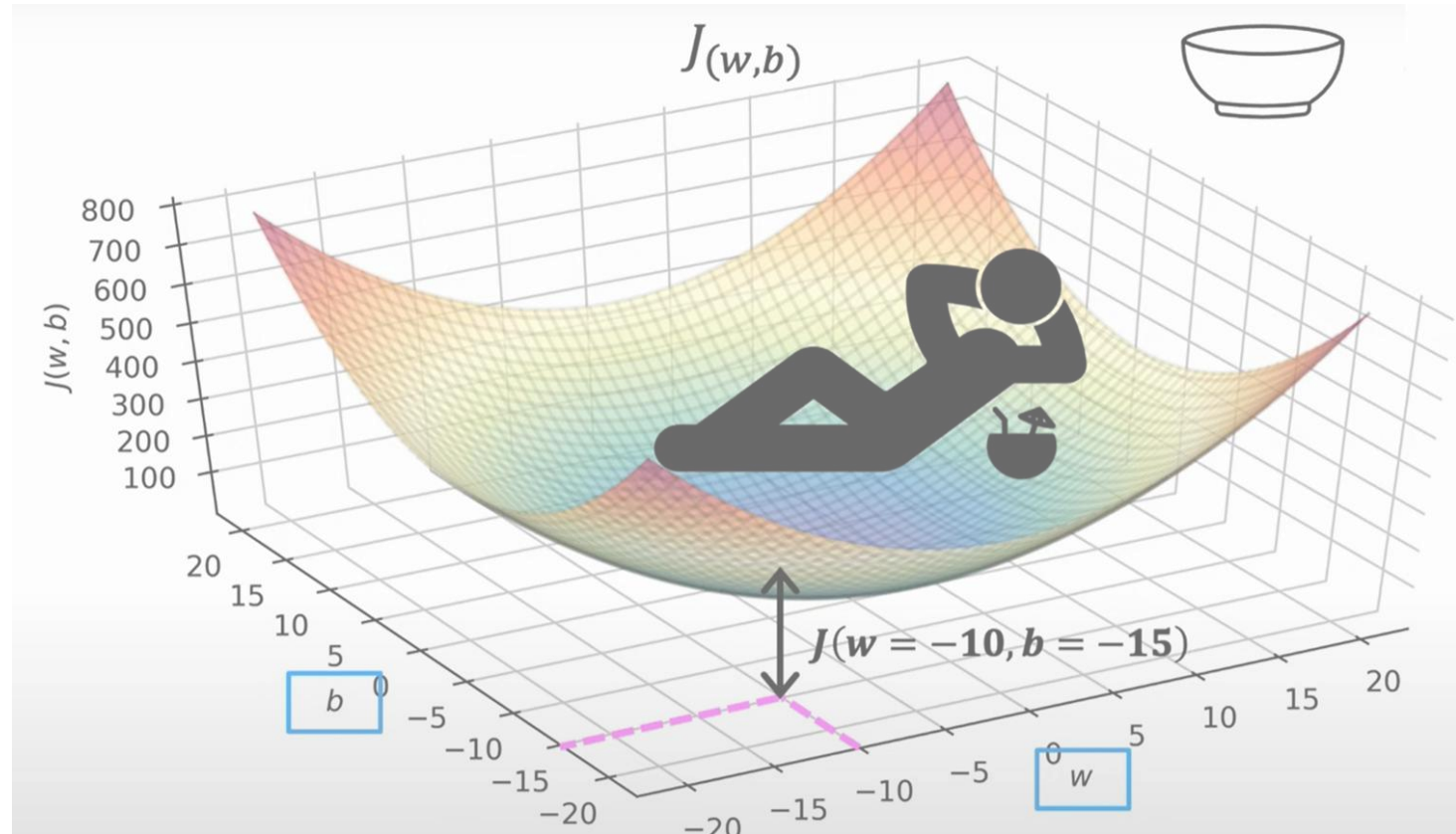
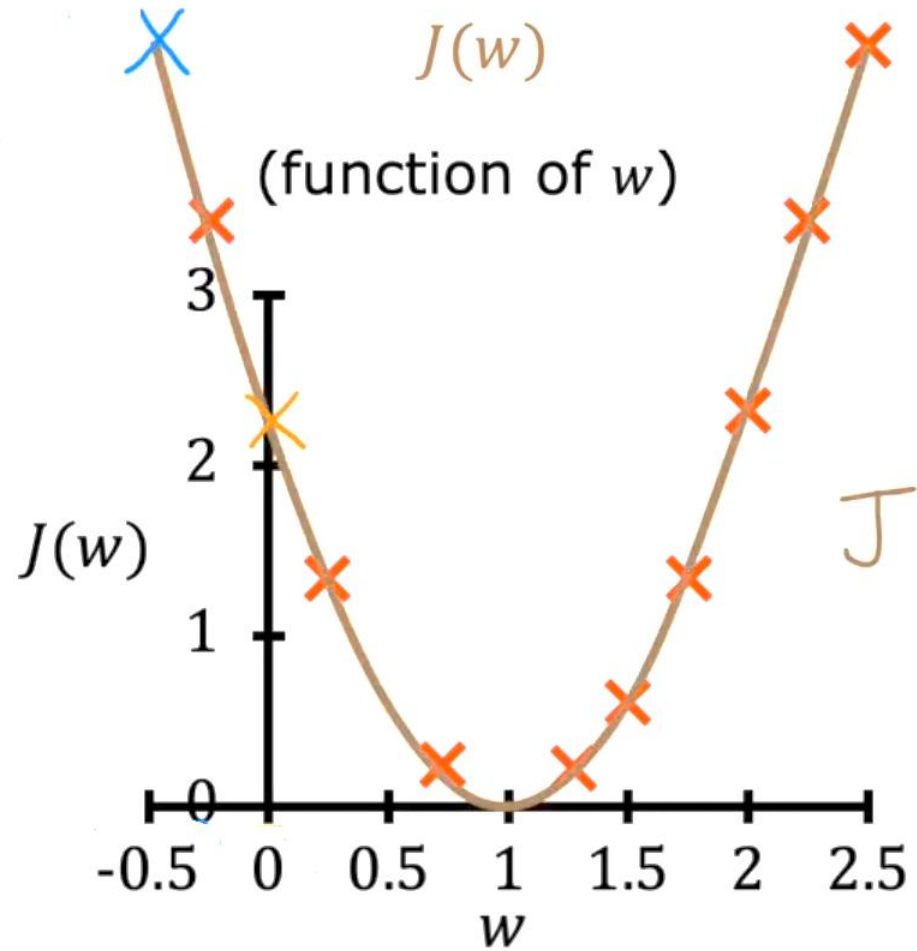
Loss Function

We basically want to find $\{w, b\}$ that minimize deviations from the predictor line

$$\text{Minimize } J(w, b) = \arg \min_{w, b} \sum_i^n (y_i - b - wx_i)^2$$



Loss Function



Solve for the Parameters Using Calculus

➤ Step 1: Define the Error Function

$$E(w, b) = \sum_{i=1}^n (y_i - b - wx_i)^2$$

➤ Step 2: Take the Partial Derivative with respect to b

Let's find $\frac{dE}{db}$:

$$\begin{aligned}\frac{dE}{db} &= \frac{d}{db} \sum_{i=1}^n (y_i - b - wx_i)^2 \\ &= \sum_{i=1}^n \frac{d}{db} (y_i - b - wx_i)^2 \\ &= \sum_{i=1}^n 2 (y_i - b - wx_i)(-1) \\ &= -2 \sum_{i=1}^n (y_i - b - wx_i)\end{aligned}$$

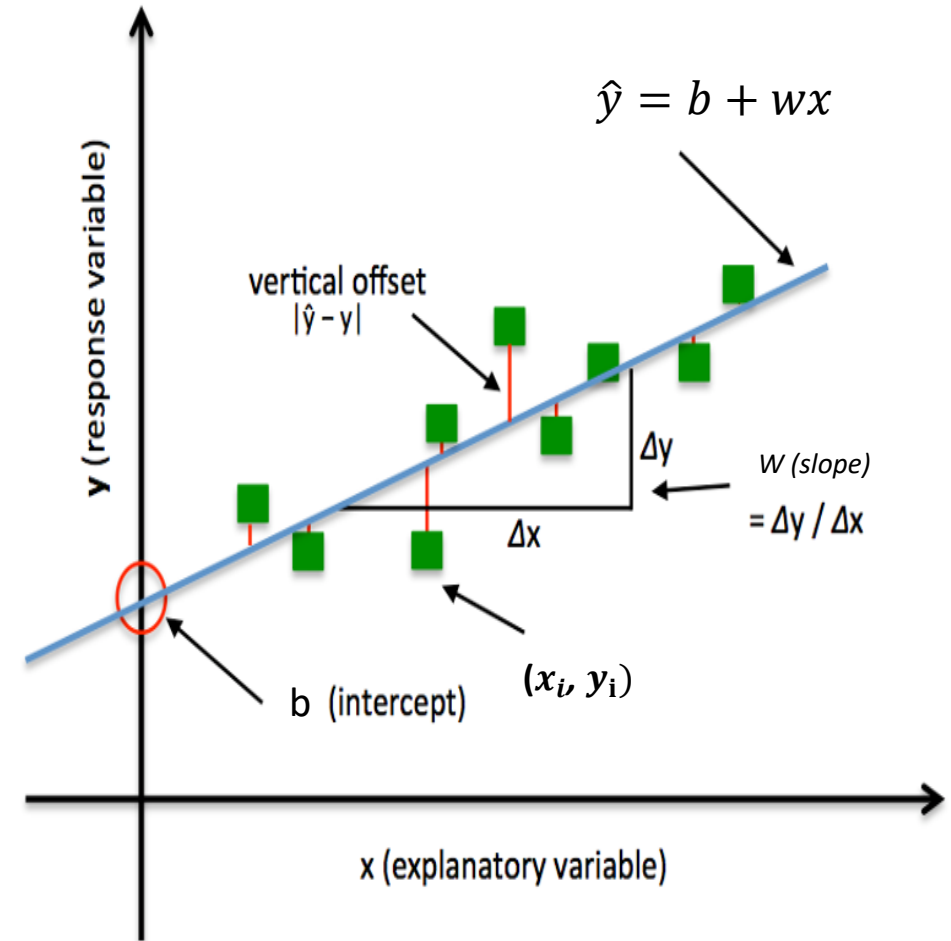
Set the derivative to zero to minimize:

$$-2 \sum_{i=1}^n (y_i - b - wx_i) = 0$$

Divide both sides by -2:

$$\sum_{i=1}^n (y_i - b - wx_i) = 0$$

— >Equation 1



Solve for the Parameters Using Calculus

Distribute the summation:

$$\Sigma y_i - \Sigma b - \Sigma w x_i = 0$$

When you sum a constant b , **n times**, it becomes:

$$\sum_{i=1}^n b = b + b + \dots + b = nb$$

Since b and w are constants:

$$\Sigma y_i - nb - w \Sigma x_i = 0$$

Solve for b :

$$nb = \Sigma y_i - w \Sigma x_i$$

$$b = \frac{1}{n} \Sigma y_i - w \frac{1}{n} \Sigma x_i$$

$$b = \bar{y} - w\bar{x} \quad \text{— >Equation 2}$$

Solve for the Parameters Using Calculus

➤ Step 3: Take the Partial Derivative with respect to w

Let's find $\frac{dE}{dw}$:

$$\begin{aligned}\frac{dE}{dw} &= \frac{d}{dw} \sum_{i=1}^n (y_i - b - wx_i)^2 \\ &= \sum_{i=1}^n \frac{d}{dw} (y_i - b - wx_i)^2 \\ &= \sum_{i=1}^n 2 (y_i - b - wx_i)(-x_i) \\ &= -2 \sum_{i=1}^n x_i (y_i - b - wx_i)\end{aligned}$$

Set the derivative to zero to minimize:

$$-2 \sum_{i=1}^n x_i (y_i - b - wx_i) = 0$$

Divide both sides by -2:

$$\sum_{i=1}^n x_i (y_i - b - wx_i) = 0 \quad \text{— >Equation 3}$$

Solve for the Parameters Using Calculus

From equation 3

$$\sum_{i=1}^n x_i(y_i - b - wx_i) = 0$$

Substitute $b = \bar{y} - w\bar{x}$

$$\sum_{i=1}^n x_i(y_i - (\bar{y} - w\bar{x}) - wx_i) = 0$$

$$\sum x_i y_i - \sum x_i (\bar{y} - w\bar{x}) - w \sum x_i^2 = 0$$

$$\sum x_i y_i - \sum x_i \bar{y} + \sum x_i w\bar{x} - w \sum x_i^2 = 0$$

$$\sum x_i y_i - \sum x_i \frac{\sum y_i}{n} + \sum x_i w \frac{\sum x_i}{n} - w \sum x_i^2 = 0$$

$$w \left[\sum x_i^2 - \frac{(\sum x_i)^2}{n} \right] = \sum x_i y_i - \frac{\sum x_i \sum y_i}{n}$$

$$w \left[n \sum x_i^2 - (\sum x_i)^2 \right] = n \sum x_i y_i - \sum x_i \sum y_i$$

The closed-form solution:

$$W = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$$

$$b = \bar{y} - w\bar{x}$$

Generalizing the Model (Vector/Matrix Form)

Given:

Training data with n input-output pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, $\mathbf{x}_i \in \mathbb{R}^D$, $y_i \in \mathbb{R}$

We model the output as:

$$y_n \approx f(\mathbf{x}_i) = \mathbf{w}^\top \mathbf{x}_i \quad (i = 1, 2, \dots, n)$$

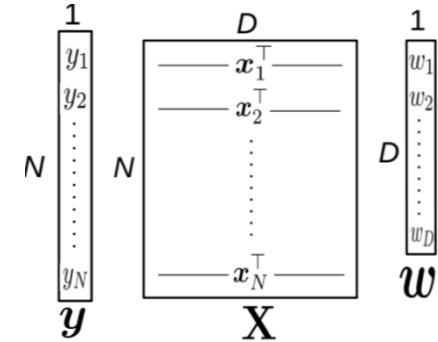
In matrix-vector notation:

$$\mathbf{y} \approx \mathbf{X}\mathbf{w}$$

The **goal** is to find $\mathbf{w}$ that **minimizes the total loss** over the dataset:

$$L(\mathbf{w}) = \sum_{i=1}^n \ell(y_i, \mathbf{w}^\top \mathbf{x}_i)$$

$\ell(y_i, \mathbf{w}^\top \mathbf{x}_i)$ measures the prediction error or “loss” or “deviation” of the model on a single training input $(\mathbf{x}_i, y_i)$



Linear Regression as Solving System of Linear Equations

Linear regression can be viewed as solving a system of linear equations:

The model

$$\mathbf{y} \approx \mathbf{X}\mathbf{w}$$

Where:

- $\mathbf{X} \in \mathbb{R}^{N \times D}$: input matrix (features);
- $\mathbf{w} \in \mathbb{R}^{D \times 1}$: weights (unknowns);
- $\mathbf{y} \in \mathbb{R}^{N \times 1}$: output (target values)

• Training Data Representation

For each training example (with D features), we can write:

$$y_n = x_{n1}w_1 + x_{n2}w_2 + \dots + x_{nD}w_D$$

So, you get n equations and D unknowns.

Linear Regression as Solving System of Linear Equations

Overdetermined vs. Underdetermined Systems

• Usually, $n \neq D$:

- Overdetermined: $n > D \rightarrow$ more equations than unknowns
- Underdetermined: $n < D \rightarrow$ more unknowns than equations

$$E = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

$$E = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$E = \mathbf{y}^T \mathbf{y} - (\mathbf{X}\mathbf{w})^T \mathbf{y} - \mathbf{y}^T (\mathbf{X}\mathbf{w}) + (\mathbf{X}\mathbf{w})^T (\mathbf{X}\mathbf{w})$$

Since scalars are equal to their transpose:

$$\mathbf{y}^T \mathbf{X}\mathbf{w} = (\mathbf{X}\mathbf{w})^T \mathbf{y}$$

Therefore,

$$E = \mathbf{y}^T \mathbf{y} - 2 (\mathbf{X}\mathbf{w})^T \mathbf{y} + (\mathbf{X}\mathbf{w})^T (\mathbf{X}\mathbf{w})$$

Now differentiate with respect to $\mathbf{w}$

$$\frac{dE}{d\mathbf{w}} = \frac{d}{d\mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2 (\mathbf{X}\mathbf{w})^T \mathbf{y} + (\mathbf{X}\mathbf{w})^T (\mathbf{X}\mathbf{w}))$$

$$\frac{dE}{d\mathbf{w}} = 0 - 2 \mathbf{X}^T \mathbf{y} + 2 \mathbf{X}^T \mathbf{X} \mathbf{w}$$

To find the minimum of the cost function, we set the gradient equal to zero.

$$0 - 2 \mathbf{X}^T \mathbf{y} + 2 \mathbf{X}^T \mathbf{X} \mathbf{w} = 0$$

$$\Rightarrow \mathbf{X}^T \mathbf{y} = \mathbf{w} \mathbf{X}^T \mathbf{X}$$

$$\Rightarrow \mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Linear Regression: Finding the Best Fit Line

To find the best fit line that minimizes the **Sum of Squared Errors (SSE)**, two main methods are used:

1. OLS (Ordinary Least Squares) – Statistical Approach

Goal: Minimize the sum of squared differences between predicted and actual values.

Formula derivation:

Given n data points: $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$

Step 1: Define the Cost Function (SSE)

$$SSE = \sum (y_i - \hat{y}_i)^2 = J(m, b) = \sum_{i=1}^n (y_i - (mx_i + b))^2$$

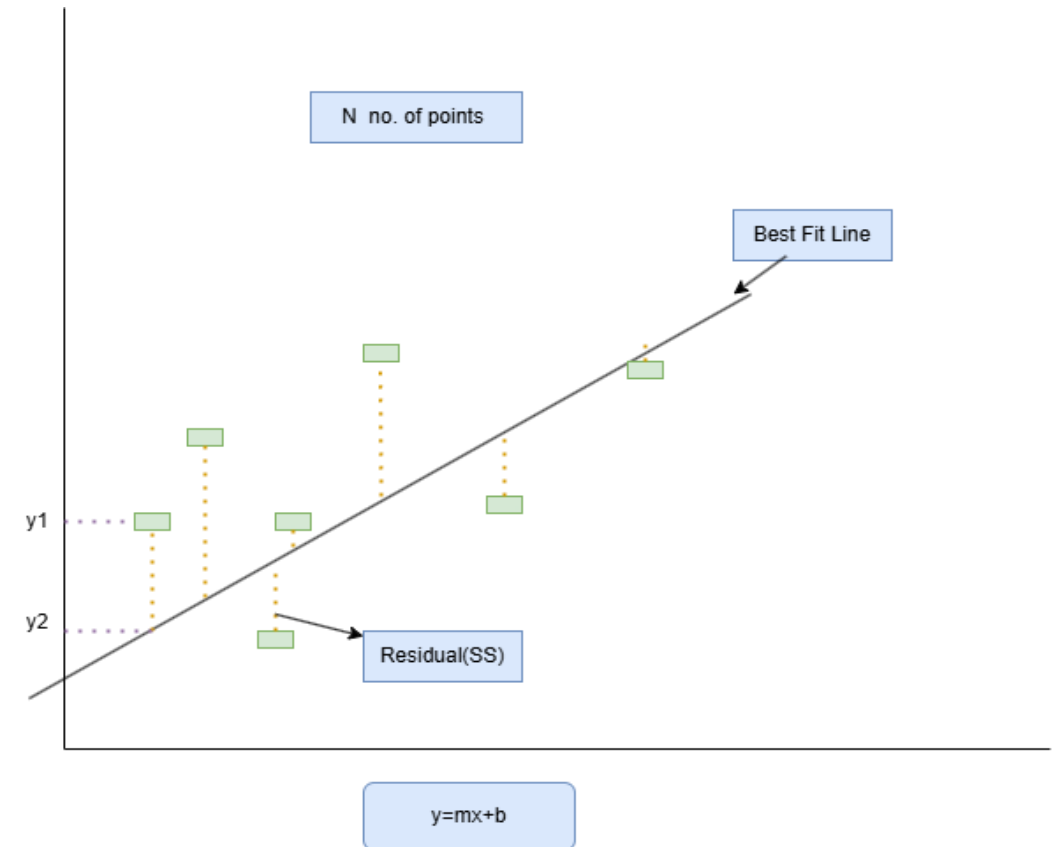
Step 2: Take Partial Derivatives

Partial derivative with respect to m:

$$\frac{\partial J}{\partial m} = \sum_{i=1}^n -2x_i(y_i - mx_i - b)$$

Partial derivative with respect to b:

$$\frac{\partial J}{\partial b} = \sum_{i=1}^n -2(y_i - mx_i - b)$$



Ordinary Least Squares

Step 3: Set Derivatives to Zero (Minimization Condition)

To minimize the error, set:

$$\frac{\partial J}{\partial m}=0 \quad \text{and} \quad \frac{\partial J}{\partial b}=0$$

Solving:

Equation (1):
$$\sum x_i(y_i - mx_i - b)=0$$

Equation (2):
$$\sum (y_i - mx_i - b)=0$$

Step 4: Solve the System of Equations

From Equation (2):

$$\sum y_i - m \sum x_i - nb = 0 \Rightarrow b = \frac{\sum y_i - m \sum x_i}{n} \Rightarrow b = \bar{y} - m\bar{x}$$

Ordinary Least Squares

Step 5: Plug into Equation (1) to Find m

$$\sum x_i y_i - m \sum x_i^2 - b \sum x_i = 0$$

Substitute $b = \bar{y} - m\bar{x}$ and simplify to get the formula for m:

$$m = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

Final Formulas:

Slope (m):

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Intercept (b):

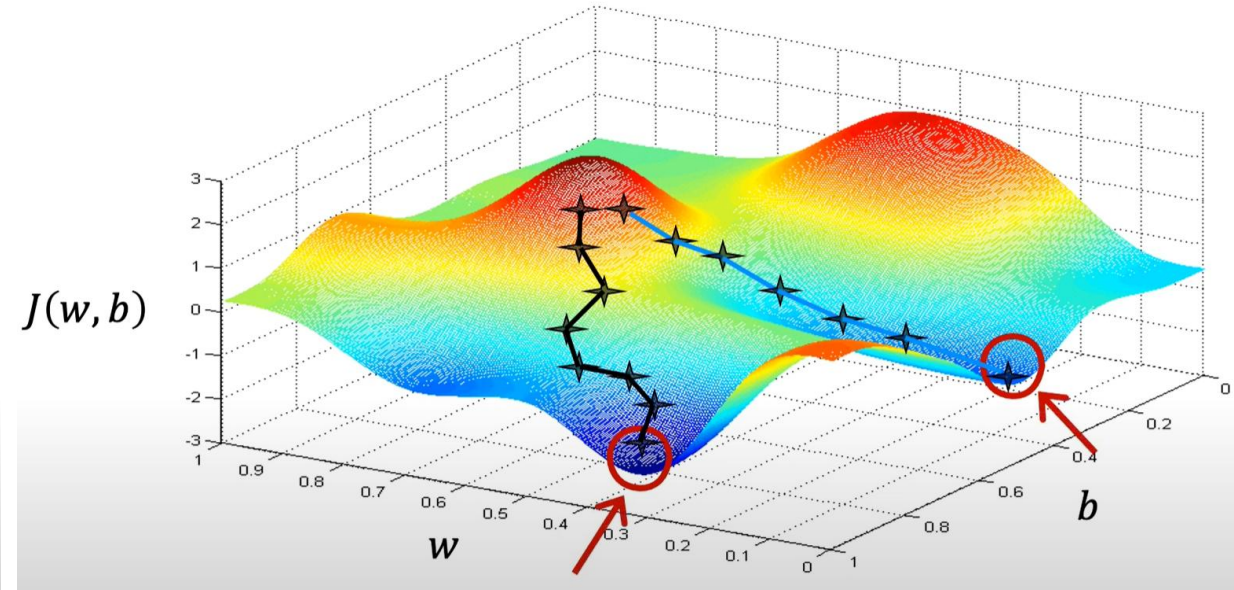
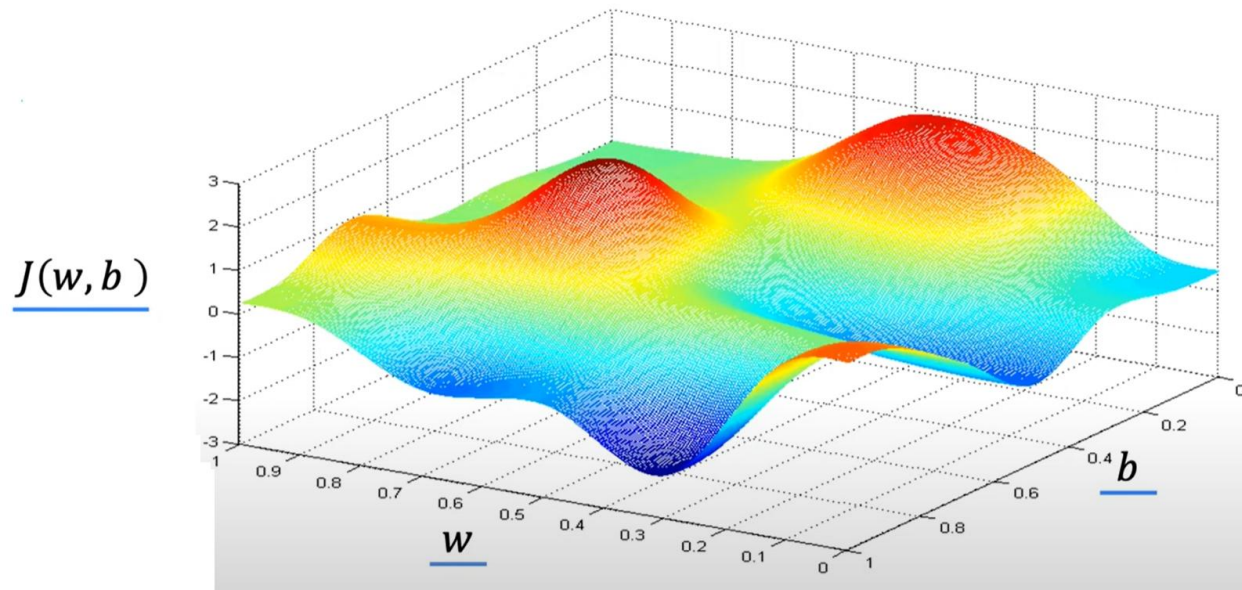
$$b = \bar{y} - m\bar{x}$$

Ordinary Least Squares

- Intuition:
OLS computes the best-fit line by minimizing the sum of squared errors using a direct algebraic solution.
It finds the exact values of coefficients analytically without iteration.
- Fast and efficient for:
Small to medium-sized datasets where the number of features isn't too high.
- Used in:
 - Simple and multiple linear regression
 - Quick statistical analysis and inference
 - Libraries: scikit-learn, statsmodels, Excel (trendline)

2. Gradient Descent – Calculus-Based Optimization

- **Objective:** Minimize $J(w, b)$
- Outline of Gradient Descent
 - Start with some w, b
 - Keep changing w, b to reduce $J(w, b)$
 - Until we settle at a minima or near minima



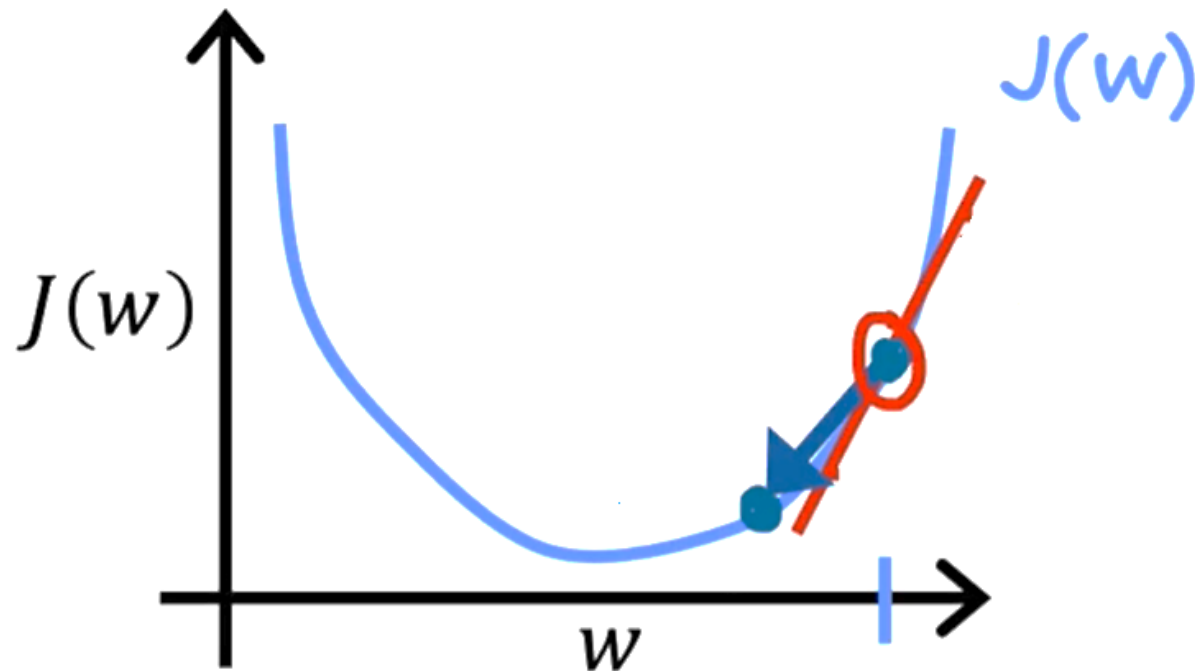
Gradient Descent Algorithm

- Repeat until convergence
- Simultaneously update w and b as
 - $w = w - \alpha \frac{\partial}{\partial w} J(w, b)$
 - $b = b - \alpha \frac{\partial}{\partial b} J(w, b)$
 - α is the learning rate that controls how big or small the step

- Simultaneous update??
 - $temp_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$
 - $temp_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$
 - $w = temp_w$
 - $b = temp_b$

Gradient Descent - Intuition

- Assume $b = 0$. **Objective:** Minimize $J(w)$
- Gradient $\rightarrow$ Slope $\rightarrow$ Derivative



$$w = w - \alpha \frac{\partial}{\partial w} J(w)$$

$$w = w - \alpha. (\text{positive number})$$

So, w is decremented to reduce $J(w)$

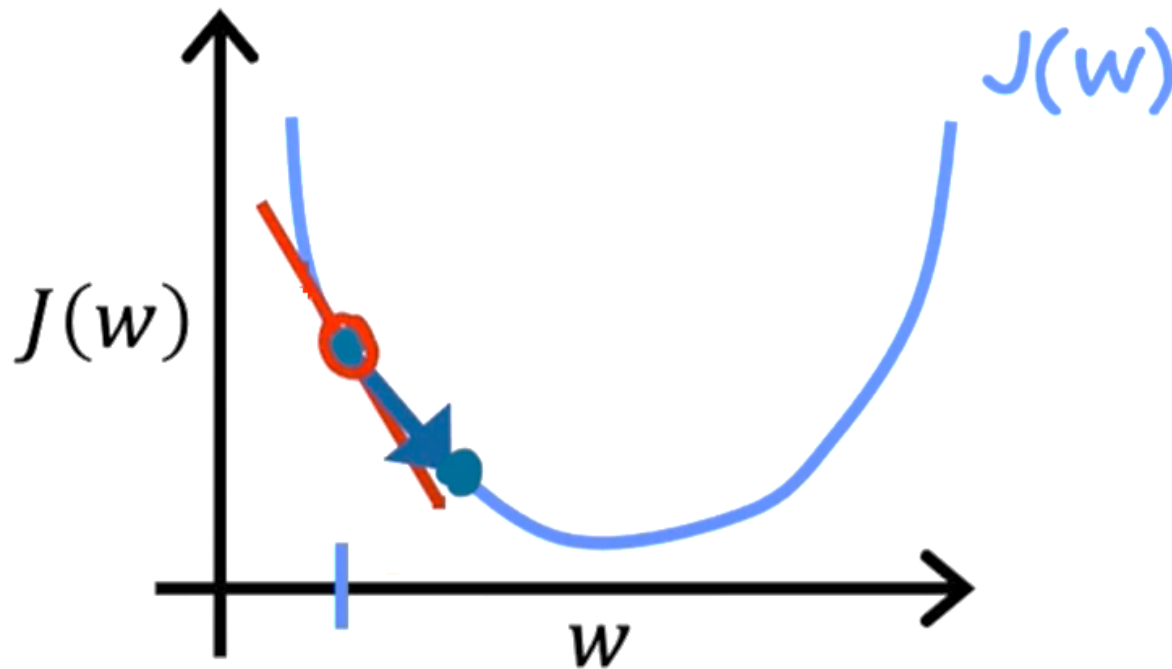
Gradient Descent - Intuition

- Assume $b = 0$. **Objective:** Minimize $J(w)$
- Gradient $\rightarrow$ Slope $\rightarrow$ Derivative

$$w = w - \alpha \frac{\partial}{\partial w} J(w)$$

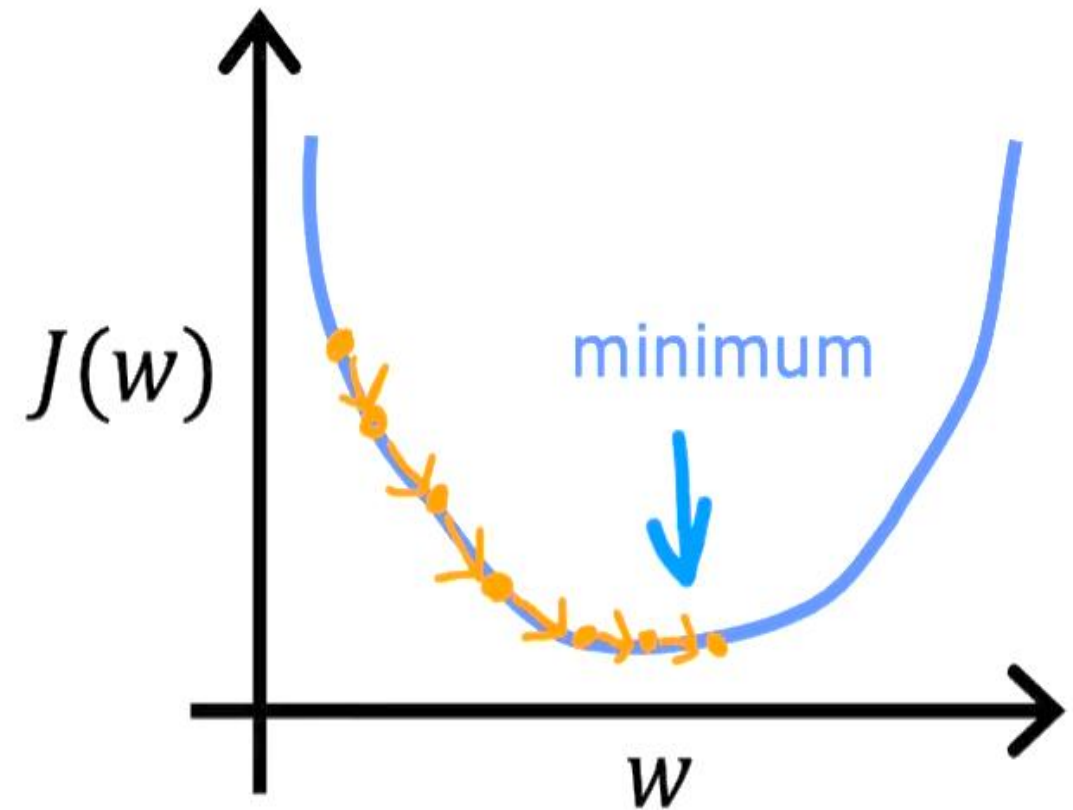
$$w = w - \alpha \cdot (\text{negative number})$$

So, w is incremented to reduce $J(w)$



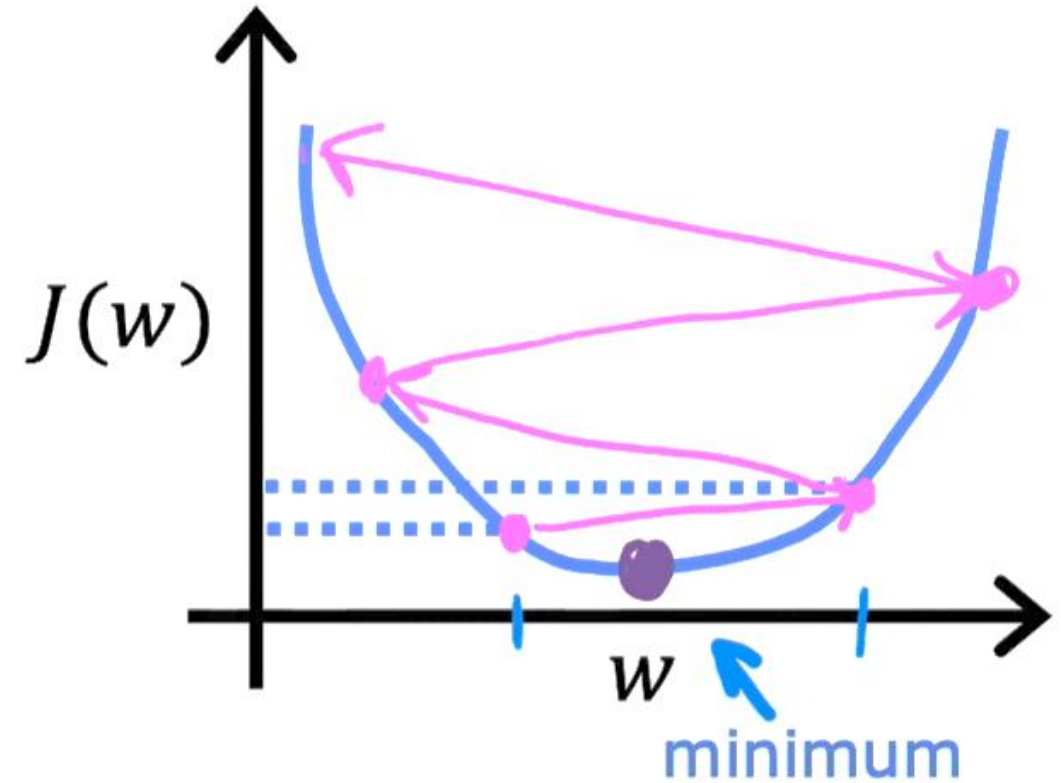
Gradient Descent – Impact of α

- $w = w - \alpha \frac{\partial}{\partial w} J(w)$
- If α is too small, Gradient descent
 - may be very slow



Gradient Descent – Impact of α

- $w = w - \alpha \frac{\partial}{\partial w} J(w)$
- If α is too large, Gradient descent
 - may overshoot
 - may never reach minimum
 - may fail to converge



Gradient for Linear Regression

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$$w = w - \alpha \frac{\partial}{\partial w} J(w, b) \Rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w, b) \Rightarrow \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

}

Gradient for Linear Regression

Derivation of Cost Function w.r.t. w

$$\begin{aligned}\frac{\partial}{\partial w} J(w, b) &= \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 \\&= \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2 \\&= \frac{1}{2m} \sum_{i=1}^m 2 \cdot (wx^{(i)} + b - y^{(i)}) \cdot x^{(i)} \\&= \frac{1}{m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cdot x^{(i)} \\&= \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}\end{aligned}$$

Gradient for Linear Regression

Derivation of Cost Function w.r.t. b

$$\frac{\partial}{\partial b} J(w, b) = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

$$= \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

$$= \frac{1}{2m} \sum_{i=1}^m 2 \cdot (wx^{(i)} + b - y^{(i)})$$

$$= \frac{1}{m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})$$

$$= \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

Example:

Goal: Iteratively adjust m and b to minimize the cost function

Example:

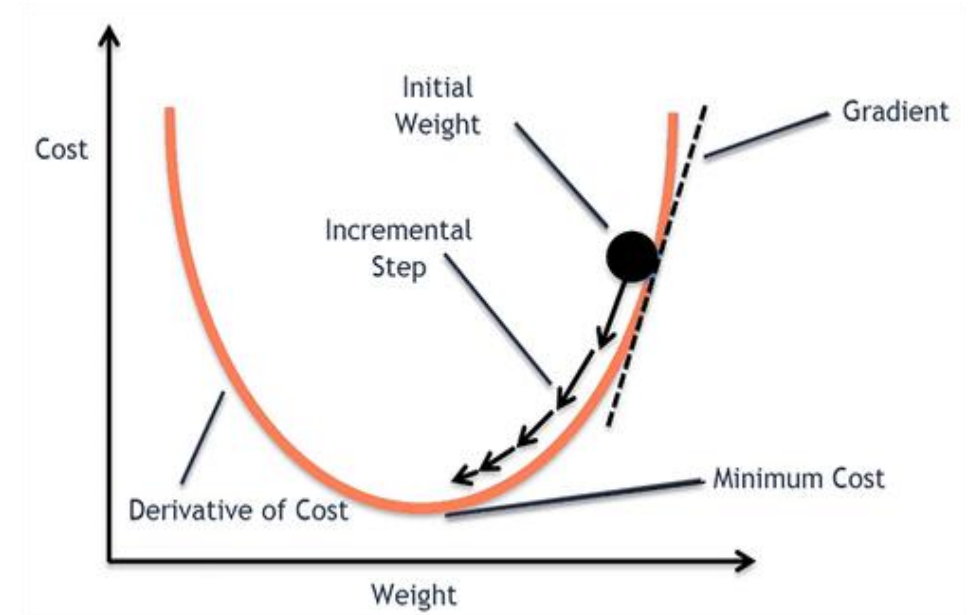
Given Data Points:

- $x_1 = 8, y_1 = 15$
- $x_2 = 18, y_2 = 30$
- $x_3 = 24, y_3 = 25$

Step 1: Define Hypothesis

Since $m=1$, the hypothesis is:

$$\hat{y}_i = x_i + b$$



Example:

Step 2: Define Cost Function (SSE - Sum of Squared Errors)

$$SS(b) = \sum_{i=1}^3 (x_i + b - y_i)^2$$

$$SS(b) = (8 + b - 15)^2 + (18 + b - 30)^2 + (24 + b - 25)^2$$

$$SS(b) = (b - 7)^2 + (b - 12)^2 + (b - 1)^2$$

Step 3: Compute Gradient of SS with respect to b

$$SS(b) = (b - 7)^2 + (b - 12)^2 + (b - 1)^2$$

Differentiate with respect to b:

$$\frac{dSS}{db} = 2(b - 7) + 2(b - 12) + 2(b - 1) = 2[3b - (7 + 12 + 1)] = 2(3b - 20) = 6b - 40$$

Example:

Step 4: Gradient Descent Step

Let's use:

- Initial $b=0$
- Learning rate $\alpha=0.1$

Iteration 1:

$$ss'(b) = 6(0) - 40 = -40$$

$$stepsize = -40 \times 0.1 = -4$$

$$b_{\text{new}} = b_{\text{old}} - stepsize = 0 - (-4) = 4$$

Iteration 2:

$$b = 4; \Rightarrow ss'(b) = 6(4) - 40 = 24 - 40 = -16$$

$$stepsize = -16 \times 0.1 = -1.6$$

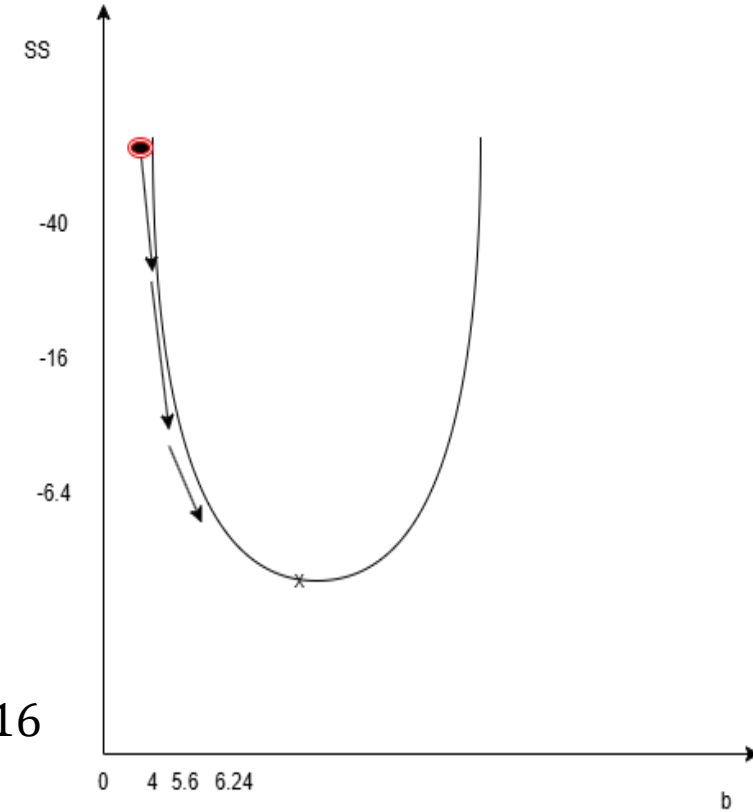
$$b_{\text{new}} = 4 - (-1.6) = 5.6$$

Iteration 3:

$$b = 5.6; \Rightarrow ss'(b) = 6(5.6) - 40 = 33.6 - 40 = -6.4$$

$$stepsize = -6.4 \times 0.1 = -0.64$$

$$b_{\text{new}} = 5.6 - (-0.64) = 6.24$$



Example:

Typically, we stop when the gradient ($SS'(b)$) becomes very small, such as:

$$|SS'(b)| < \epsilon$$

Where ϵ is a small threshold like 0.01 or 0.001.

We are using:

$$SS'(b) = 6b - 40$$

Let's solve for when $SS'(b) \approx 0$:

Set $SS'(b) = 0$

$$6b - 40 = 0 \Rightarrow b = 40/6 = 6.67$$

So the gradient descent will converge as $b \rightarrow 6.67$

Gradient for Linear Regression

➤ Intuition:

Gradient Descent is an iterative optimization algorithm that adjusts model parameters gradually to minimize the cost (error).

It uses the slope (gradient) of the cost function to update weights step by step toward the minimum.

➤ Fast and efficient for:

Large datasets and situations where an exact solution is hard or expensive to compute.

➤ Used in:

- Training linear and logistic regression models (with large data)
- Training neural networks and deep learning models
- Libraries: TensorFlow, PyTorch, scikit-learn (SGDRegressor, SGDClassifier)

LR with multiple variables

Previous notation

Vector notation

Parameters

$$w_1, \dots, w_n$$
$$b$$

$$\vec{w} = [w_1 \quad \dots \quad w_n]$$

b a number

Model

$$f_{\vec{w},b}(\vec{x}) = w_1 x_1 + \dots + w_n x_n + b$$

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Cost function

$$J(w_1, \dots, w_n, b)$$

$$J(\vec{w}, b)$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(w_1, \dots, w_n, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(w_1, \dots, w_n, b)$$

}

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

LR with multiple variables

Gradient descent

One feature

repeat {

$$w = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$$

$$\frac{\partial}{\partial w} J(w, b)$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

simultaneously update w, b

}

n features ($n \geq 2$)

repeat {

$$w_1 = w_1 - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_1^{(i)}$$

$\vdots$

$$\frac{\partial}{\partial w_1} J(\vec{w}, b)$$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_n^{(i)}$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

simultaneously update

w_j (for $j = 1, \dots, n$) and b

}

Stochastic Gradient Descent

- **Batch GD:** Each iteration updates $n+1$ variables and each update looks at all the training examples.
 - Batch GD may be quite slow for very large datasets.

- **Stochastic GD:**

```
for i=1 to m{
```

$$w_j = w_j - \alpha (f_{w,b}(x^{(i)}) - y^{(i)})x_j^{(i)}$$

$$b = b - \alpha (f_{w,b}(x^{(i)}) - y^{(i)})$$

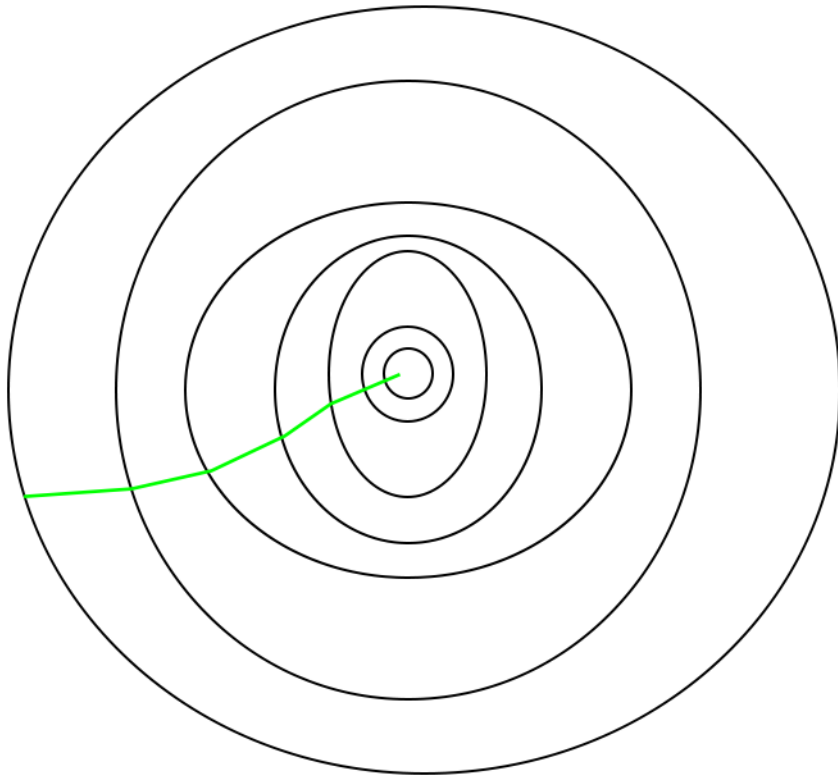
simultaneously update

w_j (for $j = 1, \dots, n$) and b

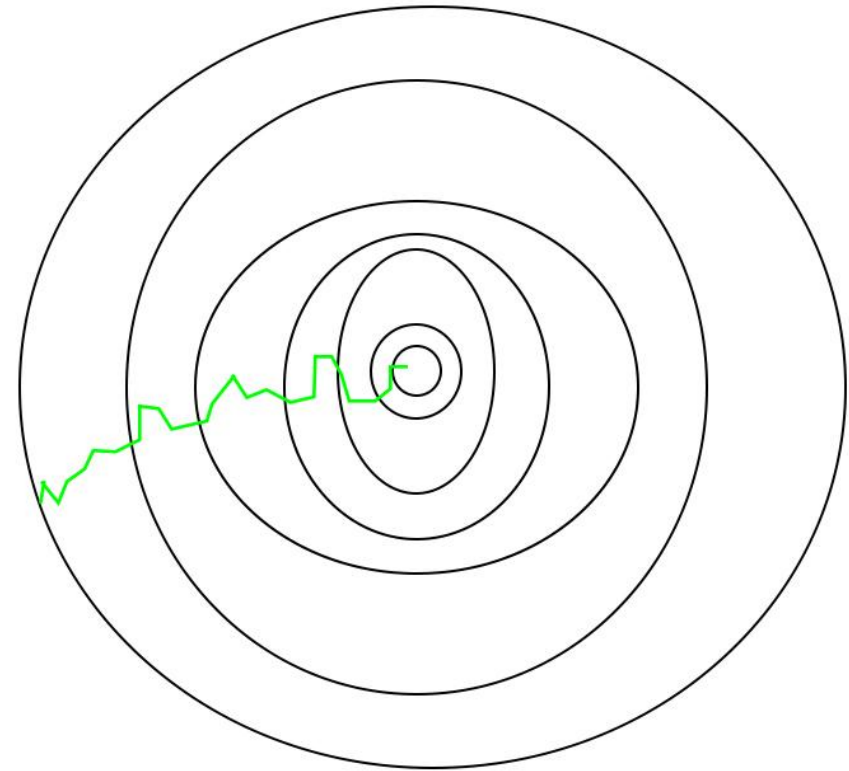
```
}
```

Batch vs. Stochastic GD

Each step in Stochastic GD may not be smooth as it sees only one training example in each iteration.
Stochastic GD may not reach the minima rather it may oscillate around the minima.



Batch Gradient Descent



Stochastic Gradient Descent

Ways to Control Overfitting

Regularization-Based Methods

- Use norms like L1 and L0 to shrink weights:

L1 Regularization: Makes many weights zero → helps in feature selection.

$$\|\mathbf{w}\|_1 = \sum_{d=1}^D |w_d|$$

L0 Regularization: Counts non-zero weights → gives very sparse models → not a true norm → **non-convex**

$$\|\mathbf{w}\|_0 = \text{#nnz}(\mathbf{w})$$

- Useful when you have lots of features, but many are irrelevant.

Non-Regularization Methods

- **Early stopping**: Stop training when validation accuracy is good.
- **Dropout**: Randomly ignore some weights during training.
- **Add noise to inputs**: Helps the model generalize better.

How do we ensure generalization?

We minimized the objective $L(\mathbf{w}) = \sum_{n=1}^N (\mathbf{y}_n - \mathbf{w}^\top \mathbf{x}_n)^2$ w.r.t. $\mathbf{w}$, and obtained

$$\mathbf{w}_{LS} = (\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^\top)^{-1} (\sum_{n=1}^N \mathbf{y}_n \mathbf{x}_n) = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

While minimizing training loss is important, it does not guarantee good performance on new/unseen data due to the following issues:

- Problem: The matrix $\mathbf{X}^\top \mathbf{X}$ may not be invertible
 - This may lead to non-unique solutions for $\mathbf{w}_{opt}$
- Problem: Overfitting since we only minimized loss defined on training data
 - Weights $\mathbf{w} = [w_1, w_2, \dots, w_D]$ may become arbitrarily large to fit training data perfectly
 - Such weights may perform poorly on the test data

To solve this issue we will use Regularization,

We will use a Minimized regularized objective :

$$L(\mathbf{w}) + \lambda R(\mathbf{w})$$

Where:

$L(\mathbf{w})$ is the Original loss (training error)

$R(\mathbf{w})$ is called the Regularizer and measures the “magnitude” of $\mathbf{w}$

$\lambda \geq 0$: Regularization strength, controls how much we wish to regularize (needs to be tuned via cross-validation)

Regularized Least Squares (a.k.a. Ridge Regression)

Ridge Regression is a regularized version of linear regression. It adds a penalty term to the loss function to prevent overfitting by shrinking large weight values.

Regularized objective is of the form

$$L_{reg}(w) = L(w) + \lambda R(w)$$

Common Regularizer: ℓ_2 Norm (Squared Length of w)

$$R(w) = \|w\|_2^2 = w^\top w$$

Final Loss for Ridge Regression:

We minimize:

$$w_{ridge} = \arg \min_w L(w) + \lambda R(w) = \arg \min_w \sum_{n=1}^N (y_n - w^\top x_n)^2 + \lambda w^\top w$$

Why is the method called “ridge” regression?

$$w_{ridge} = (\sum_{n=1}^N x_n x_n^\top + \lambda I_D)^{-1} (\sum_{n=1}^N y_n x_n) = (X^\top X + \lambda I_D)^{-1} X^\top y$$

This works even if $X^\top X$ is not invertible, because we add λ to the diagonal, making it stable — like adding a ridge to flatten rough terrain!

Polynomial Regression

Polynomial regression is used when the relationship between the independent variable and dependent variable is **non-linear**. It fits a polynomial equation to the data.

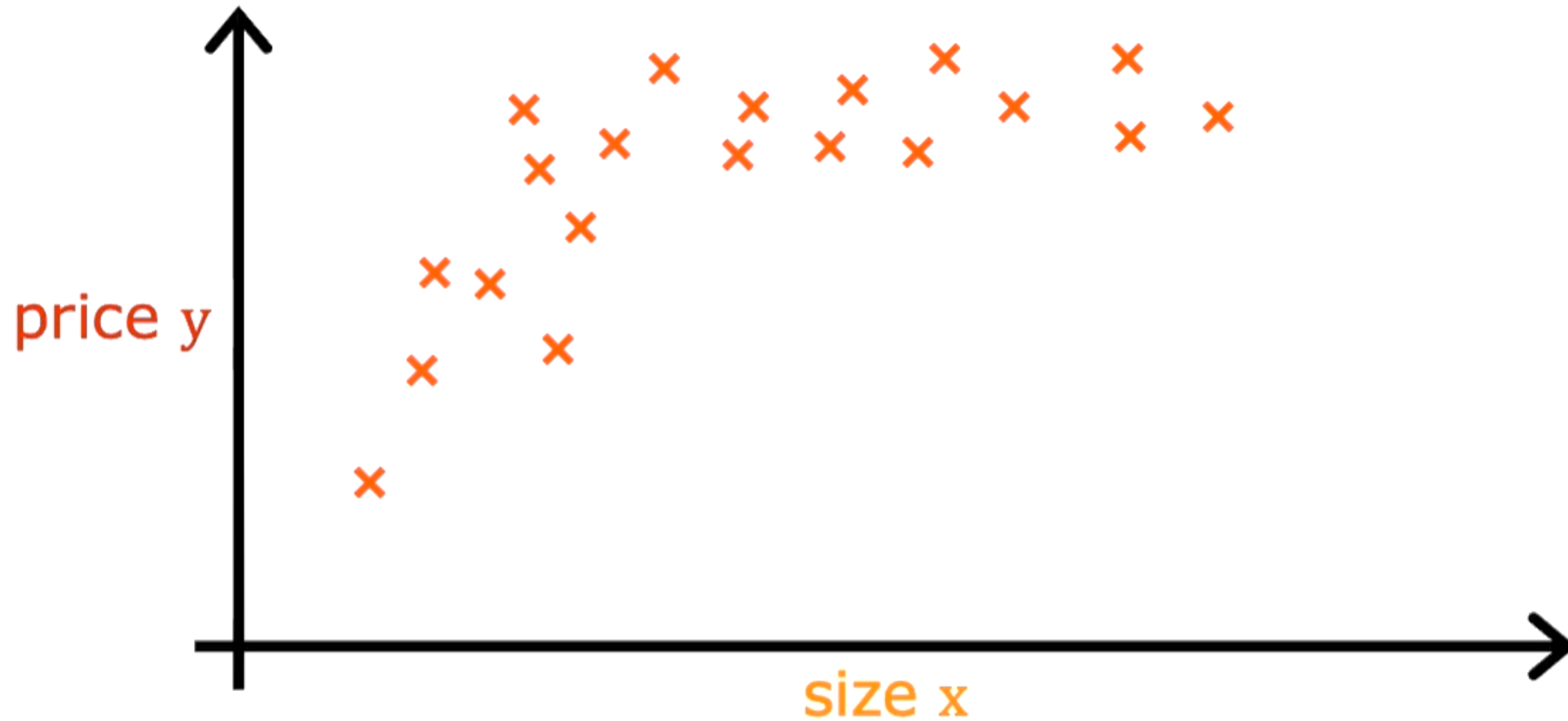
Formula:

$$y = b + w_1x + w_2x^2 + w_3x^3 + \dots + w_dx^d + \epsilon$$

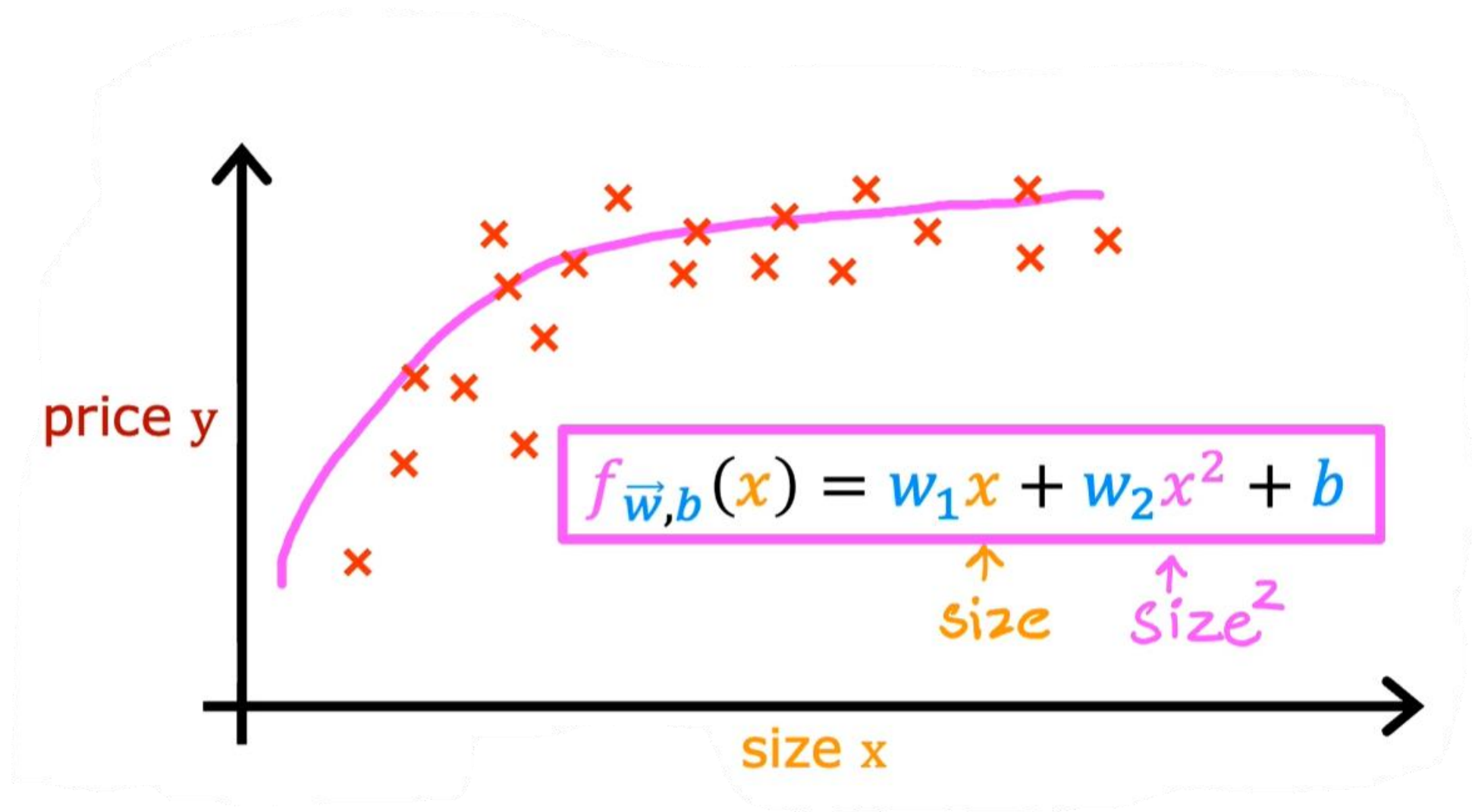
Where:

- y is the actual observed output
- x is the independent variable
- d is the degree of the polynomial
- ϵ is the error term

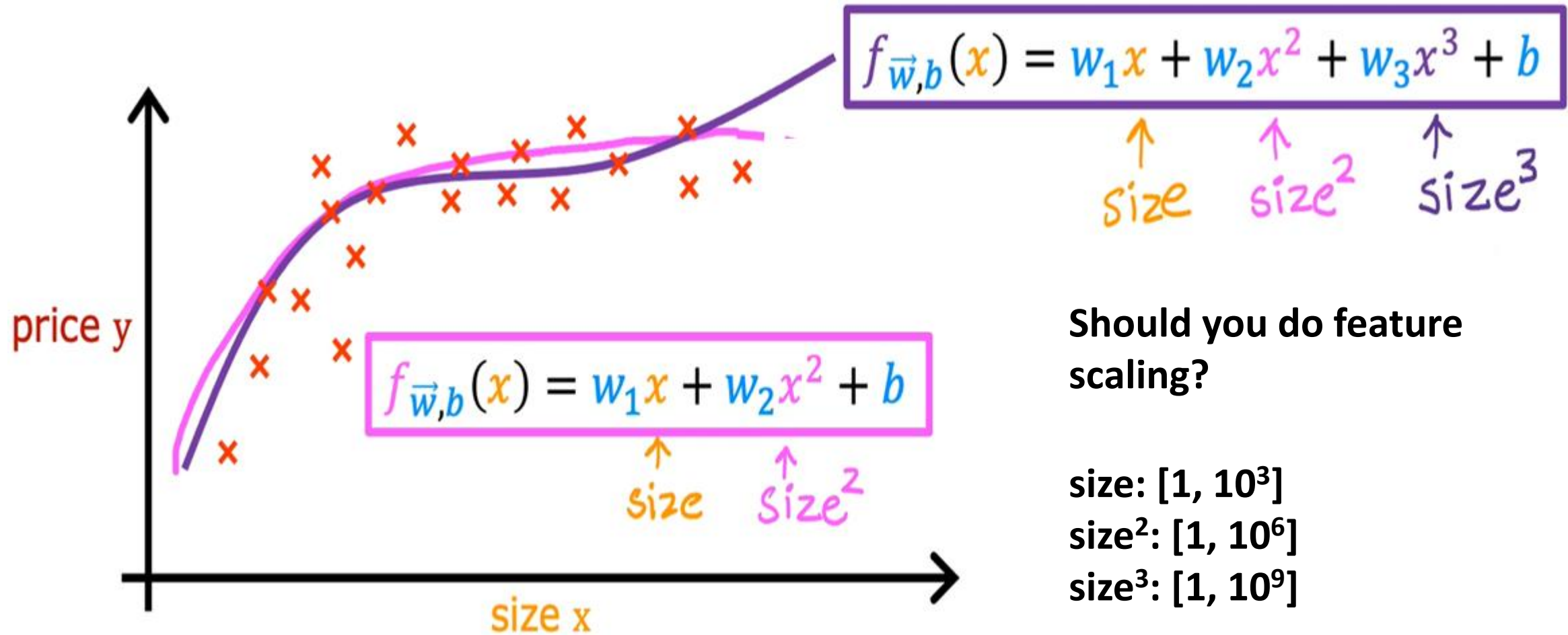
Polynomial Regression



Polynomial Regression



Polynomial Regression



Should you do feature scaling?

size: [1, 10³]

size²: [1, 10⁶]

size³: [1, 10⁹]

Polynomial Regression Model

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \dots + w_mx^m$$

This can be written as

$$y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^m w_j x^j$$

Feature Vector Representation

$$\varphi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \cdot \\ \cdot \\ \cdot \\ x^m \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \cdot \\ \cdot \\ \cdot \\ w_m \end{bmatrix}$$

The model output can be written in vector form as:

$$y_i = \varphi^T(\mathbf{x}_i) \mathbf{w}, \quad i=1,2,3,\dots,n$$

Let predicted output be $\mathbf{y}$ and desired output be $\mathbf{t}$

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \cdot \\ \cdot \\ \cdot \\ y_n \end{bmatrix} \quad \mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \\ \cdot \\ \cdot \\ \cdot \\ t_n \end{bmatrix}$$

Derivation

Define the design matrix φ as :

$$\varphi = \begin{bmatrix} \varphi^T(\mathbf{x}_1) \\ \varphi^T(\mathbf{x}_2) \\ \vdots \\ \varphi^T(\mathbf{x}_n) \end{bmatrix}$$

Define the design matrix φ as :

$$\mathbf{y} = \varphi \mathbf{w}$$

$$E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^n (t_i - y_i)^2$$

Substituting $\mathbf{y} = \varphi \mathbf{w}$

$$E(\mathbf{w}) = \frac{1}{2} (\mathbf{t} - \varphi \mathbf{w})^T (\mathbf{t} - \varphi \mathbf{w})$$

$$E(\mathbf{w}) = \frac{1}{2} [\mathbf{t}^T \mathbf{t} - \mathbf{t}^T \varphi \mathbf{w} - (\varphi \mathbf{w})^T \mathbf{t} + (\varphi \mathbf{w})^T (\varphi \mathbf{w})]$$

As scalar are equal to their transpose:

$$\mathbf{t}^T \varphi \mathbf{w} = (\varphi \mathbf{w})^T \mathbf{t}$$

$$E(\mathbf{w}) = \frac{1}{2} [\mathbf{t}^T \mathbf{t} - 2\mathbf{t}^T \varphi \mathbf{w} + (\varphi \mathbf{w})^T (\varphi \mathbf{w})]$$

Derivation

$$E(\mathbf{w}) = \frac{1}{2} [\mathbf{t}^T \mathbf{t} - 2\mathbf{t}^T \boldsymbol{\varphi} \mathbf{w} + (\boldsymbol{\varphi} \mathbf{w})^T (\boldsymbol{\varphi} \mathbf{w})]$$

To minimize the error, we differentiate $E(\mathbf{w})$ with respect to $\mathbf{w}$:

$$\frac{dE}{d\mathbf{w}} = \frac{1}{2} [0 - 2\boldsymbol{\varphi}^T \mathbf{t} + 2\boldsymbol{\varphi}^T \boldsymbol{\varphi} \mathbf{w}]$$

$$\frac{dE}{d\mathbf{w}} = -\boldsymbol{\varphi}^T \mathbf{t} + \boldsymbol{\varphi}^T \boldsymbol{\varphi} \mathbf{w}$$

For Optimal $\mathbf{W}^*$ Gradient will become Zero

$$\Rightarrow -\boldsymbol{\varphi}^T \mathbf{t} + \boldsymbol{\varphi}^T \boldsymbol{\varphi} \mathbf{w}^* = 0$$

$$\Rightarrow \boldsymbol{\varphi}^T \boldsymbol{\varphi} \mathbf{w}^* = \boldsymbol{\varphi}^T \mathbf{t}$$

$$\Rightarrow \mathbf{w}^* = (\boldsymbol{\varphi}^T \boldsymbol{\varphi})^{-1} (\boldsymbol{\varphi}^T \mathbf{t})$$

For a constant vector $\mathbf{a}$ and matrix $\mathbf{B}$:

$$\frac{dE}{d\mathbf{w}} (\mathbf{a}^T \mathbf{B} \mathbf{w}) = \mathbf{B}^T \mathbf{a}$$

Thank You